



11. MORPHOLOGICAL DILATION



Computer Vision – Image Enhancement & Morphology



(a) CONCEPT EXPLANATION

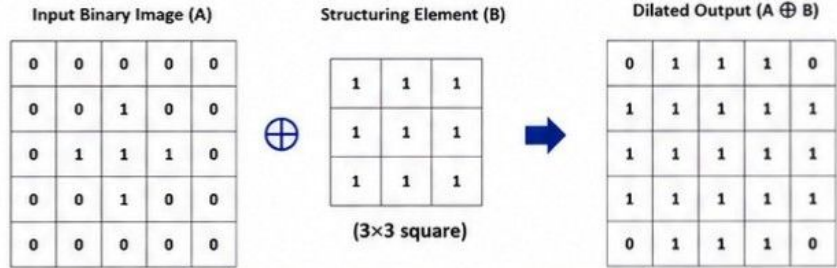
Q. (a) Explain the concept of morphological dilation.

Morphological dilation is a fundamental operation in morphological image processing. It uses a structuring element (kernel) to **expands the boundaries** of foreground (object) pixels.

It adds pixels to the boundaries of objects, **fills small gaps**, and **increases the size** of regions.

Dilation is useful for tasks like **connecting disjoint objects**, closing small holes, and enhancing object regions.

Dilation Process (Example)

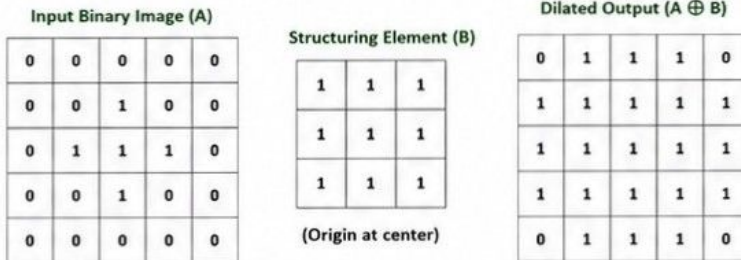


Dilation adds pixels around the object pixels, expanding its area.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 5×5 binary image and 3×3 structuring element (all ones). Perform dilation and show the resulting binary image.



Explanation (Sample Positions)			
Center Position (i, j)	Neighborhood (3×3 window)	Any 1 in Neighborhood?	Output Value
(1,1)	All 0	No	0
(1,2)	Contains 1	Yes	1
(2,1)	Contains 1	Yes	1
(2,2)	Contains 1	Yes	1
(3,3)	Contains 1	Yes	1
(4,1)	Contains 1	Yes	1

What does the result mean?

- All 1-valued pixels in the input image expand to their surrounding neighborhood.
- The object region becomes larger.
- Small gaps between objects can be filled.
- Boundaries are thickened.

Final Answer: Dilated Output (A ⊕ B) =

0	1	1	1	0
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
0	1	1	1	0



(c) GOOGLE COLAB PYTHON PROGRAM

```
# Morphological Dilation Example
import numpy as np
from scipy.ndimage import binary_dilation

# Input binary image (5x5)
A = np.array([[0, 0, 0, 0, 0],
              [0, 0, 1, 0, 0],
              [0, 1, 1, 1, 0],
              [0, 0, 1, 0, 0],
              [0, 0, 0, 0, 0]], dtype=int)

# Structuring element (3x3 square kernel)
B = np.ones((3, 3), dtype=int)

# Perform dilation
A_dilated = binary_dilation(A, structure=B)

print("Input Binary Image (A):")
print(A)
print("\nStructuring Element (B):")
print(B)
print("\nDilated Output (A ⊕ B):")
print(A_dilated.astype(int))
```

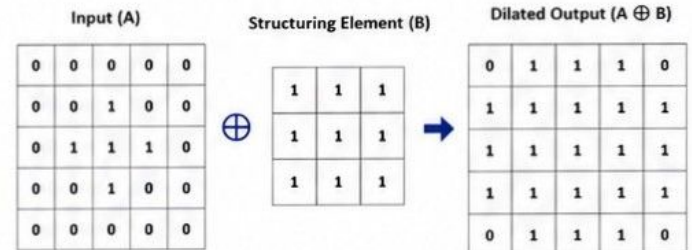
Output (Sample)

```
Input Binary Image (A):
[[0 0 0 0 0]
 [0 0 1 0 0]
 [0 1 1 1 0]
 [0 0 1 0 0]
 [0 0 0 0 0]]

Structuring Element (B):
[[1 1 1]
 [1 1 1]
 [1 1 1]]

Dilated Output (A ⊕ B):
[[0 1 1 1 0]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [0 1 1 1 0]]
```

Visual Illustration



Note: Dilation with a 3×3 all-ones kernel adds a 1-pixel thick boundary around the object.



Observation: Morphological dilation expands the object region by adding pixels to its boundaries using the structuring element. It is widely used to connect nearby objects, fill small holes, and enhance object visibility.



12. MORPHOLOGICAL EROSION



Computer Vision – Image Enhancement & Morphology



(a) CONCEPT EXPLANATION

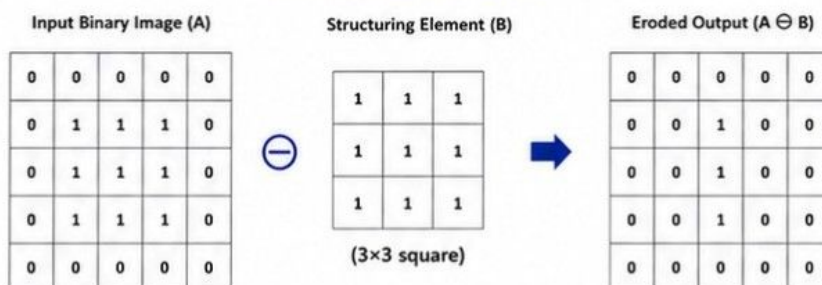
Q. (a) Explain the concept of morphological erosion.

Morphological erosion is a fundamental operation in morphological image processing. It uses a structuring element (kernel) to **shrink the boundaries** of foreground (object) pixels.

It **removes pixels** on the boundaries, eliminates small protrusions, **breaks thin connections**, and **reduces** the size of regions.

Erosion is useful for tasks like **removing noise**, **separating** connected objects, and **thinning** objects.

Erosion Process (Example)



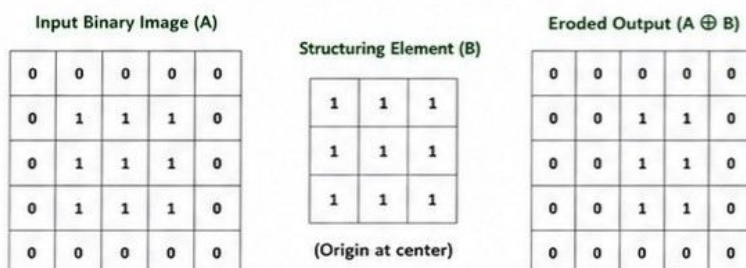
Erosion removes boundary pixels, shrinking the object region.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 5×5 binary image and 3×3 structuring element (all ones).

Perform erosion and show the resulting binary image.



Final Answer: Eroded Output (A ⊖ B) =

0	0	0	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	0	0	0

Explanation (Sample Positions)

Center Position (i, j)	3×3 Neighborhood (All ones?)	All 1's Present?	Output Value
(1,1)	All 0	No	0
(2,2)	Contains 0	No	0
(2,3)	All 1	Yes	1
(3,3)	All 1	Yes	1
(4,3)	All 1	Yes	1
(3,2)	Contains 0	No	0

What does the result mean?

- The object region shrinks because boundary pixels are removed.
- Small protrusions and thin connections are eliminated.
- The central part of the object remains.
- Erosion helps in noise removal and object separation.



(c) GOOGLE COLAB PYTHON PROGRAM

```
# Morphological Erosion Example
import numpy as np
from scipy.ndimage import binary_erosion

# Input binary image (5x5)
A = np.array([[0, 0, 0, 0, 0],
              [0, 1, 1, 1, 0],
              [0, 1, 1, 1, 0],
              [0, 1, 1, 1, 0],
              [0, 0, 0, 0, 0]], dtype=int)

# Structuring element (3x3 all ones)
B = np.ones((3,3), dtype=int)

# Perform erosion
A_eroded = binary_erosion(A, structure=B)

print("Input Binary Image (A):")
print(A)
print("\nStructuring Element (B):")
print(B)
print("\nEroded Output (A ⊖ B):")
print(A_eroded.astype(int))
```

Output (Sample)

Input Binary Image (A):

```
[[0 0 0 0 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 0 0 0 0]]
```

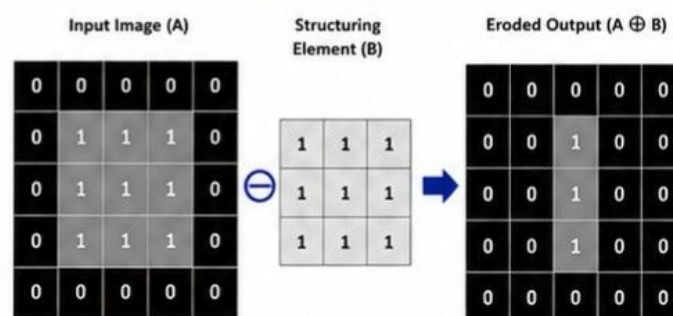
Structuring Element (B):

```
[[1 1 1]
 [1 1 1]
 [1 1 1]]
```

Eroded Output (A ⊖ B):

```
[[0 0 0 0 0]
 [0 0 1 0 0]
 [0 0 1 0 0]
 [0 0 1 0 0]
 [0 0 0 0 0]]
```

Visual Illustration



Observation:

Morphological erosion shrinks the object by removing boundary pixels using the structuring element. It is useful for noise removal, breaking thin connections, and separating connected objects.



13. IMAGE GRADIENT USING SOBEL OPERATOR



Computer Vision – Image Enhancement – Edge Detection



(a) CONCEPT EXPLANATION

Q. (a) Explain the concept of image gradient using Sobel operator.

The Sobel operator is widely used to detect edges in images. It computes the gradient (first derivative) of intensity in both horizontal (x) and vertical (y) directions using two 3×3 kernels (G_x and G_y).

The gradient magnitude indicates the strength of edges, and the gradient direction shows the orientation of edges.

$$\text{Gradient magnitude } |G| = \sqrt{G_x^2 + G_y^2}$$

$$\text{Gradient direction } \theta = \tan^{-1}(G_y / G_x)$$

Sobel Kernels

Horizontal Kernel (G_x)

-1	0	+1
-2	0	+2
-1	0	+1

Vertical Kernel (G_y)

-1	-2	-1
0	0	0
+1	+2	+1

How it works

- Convolve the image with G_x to get horizontal gradient.
- Convolve the image with G_y to get vertical gradient.
- Combine G_x and G_y to get gradient magnitude and direction.

Sobel operator smooths the image slightly and is less sensitive to noise compared to basic operators.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 3×3 image patch, compute G_x , G_y , gradient magnitude $|G|$ and direction θ (in degrees) at the center pixel using Sobel operator.

Given 3×3 Image Patch (I)

1	2	3
4	5	6
7	8	9

Sobel Kernels

G_x

-1	0	+1
-2	0	+2
-1	0	+1

G_y

-1	-2	-1
0	0	0
+1	+2	+1

Computation at Center (5)

$$G_x = \sum (I \times G_x)$$

$1 \times (-1)$	2×0	$3 \times (+1)$
$4 \times (-2)$	5×0	$6 \times (+2)$
$7 \times (-1)$	8×0	$9 \times (+1)$

$$\text{Sum} = (-1 + 0 + 3) + (-8 + 0 + 12) + (-7 + 0 + 9)$$

$$G_x = 8$$

$$G_y = \sum (I \times G_y)$$

$1 \times (-1)$	$2 \times (-2)$	$3 \times (-1)$
4×0	5×0	6×0
$7 \times (+1)$	$8 \times (+2)$	$9 \times (+1)$

$$\text{Sum} = (-1 - 4 - 3) + (0 + 0 + 0) + (7 + 16 + 9)$$

$$G_y = 24$$

Interpretation

- $G_x = 8 \rightarrow$ Change in intensity in horizontal direction.
- $G_y = 24 \rightarrow$ Change in intensity in vertical direction.
- $|G| = 25.30 \rightarrow$ Strong edge strength at the center pixel.
- $\theta = 71.57^\circ \rightarrow$ Edge is oriented $\sim 72^\circ$ from the positive x-axis.

Gradient Magnitude and Direction

$$\begin{aligned} |G| &= \sqrt{G_x^2 + G_y^2} \\ &= \sqrt{8^2 + 24^2} = \sqrt{64 + 576} \\ &= \sqrt{640} = 25.30 \end{aligned}$$

$$\begin{aligned} \theta &= \tan^{-1}\left(\frac{G_y}{G_x}\right) = \tan^{-1}\left(\frac{24}{8}\right) \\ &= \tan^{-1}(3) = 71.57^\circ \end{aligned}$$

Final Answer: $G_x = 8$, $G_y = 24$, $|G| = 25.30$, $\theta = 71.57^\circ$



(c) GOOGLE COLAB PYTHON PROGRAM

Python Code

```
import numpy as np

# Given 3x3 image patch
I = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Sobel kernels
Gx = np.array([[-1, 0, 1],
               [-2, 0, 2],
               [-1, 0, 1]])
Gy = np.array([[-1, -2, -1],
               [0, 0, 0],
               [1, 2, 1]])

# Convolution (element-wise multiplication and sum)
Gx_val = np.sum(I * Gx)
Gy_val = np.sum(I * Gy)

# Gradient magnitude and direction
G_mag = np.sqrt(Gx_val**2 + Gy_val**2)
theta = np.degrees(np.arctan2(Gy_val, Gx_val))

print("Gx =", Gx_val)
print("Gy =", Gy_val)
print("Gradient Magnitude |G| = {:.2f}".format(G_mag))
print("Gradient Direction \theta = {:.2f} degrees".format(theta))
```

Output (Sample)

Gx = 8
Gy = 24
Gradient Magnitude $|G| = 25.30$
Gradient Direction $\theta = 71.57$ degrees

Visual Illustration

Image Patch (I)

1	2	3
4	5	6
7	8	9

Gradient Components

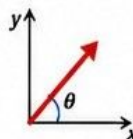
G_x

-1	0	1
-2	0	2
-1	0	1

G_y

-1	-2	-1
0	0	0
1	2	1

Edge Direction



What does it mean?

- Large $|G|$ value indicates a strong edge.
- Here, $|G| = 25.30 \rightarrow$ strong edge detected at center pixel.
- $\theta \approx 71.57^\circ$ means the edge is oriented mostly in vertical direction.
- Sobel operator is effective for detecting edges with some noise suppression.



Observation: Sobel operator computes image gradient in horizontal and vertical directions. The gradient magnitude and direction help in identifying the strength and orientation of edges in the image.

14. CANNY EDGE DETECTION ALGORITHM



Computer Vision – Image Enhancement – Edge Detection



(a) CONCEPT EXPLANATION

Q. (a) Explain Canny Edge Detection algorithm.

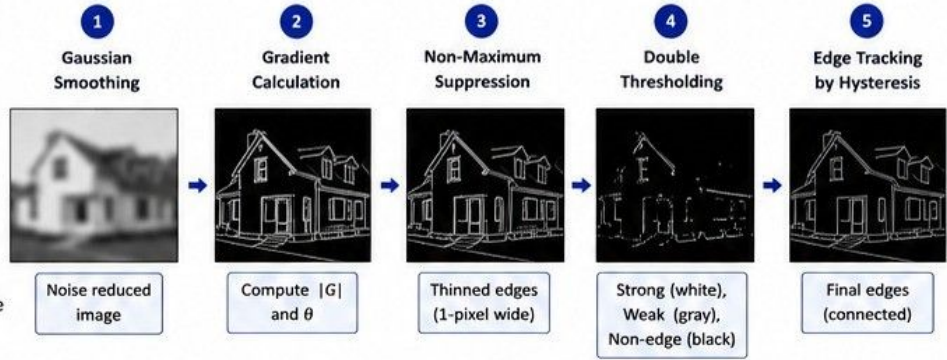
Canny Edge Detection is a multi-stage algorithm used to detect strong and thin edges in images.

It is optimal in the sense of **good detection**, **good localization**, and **single response**.

Steps in Canny Edge Detection:

1. **Gaussian Smoothing** – reduces noise.
2. **Gradient Calculation** – compute intensity gradients (G_x , G_y), gradient magnitude ($|G|$) and direction (θ).
3. **Non-Maximum Suppression** – thins edges to 1-pixel width.
4. **Double Thresholding** – classifies pixels as strong, weak or non-edge using high (T_H) and low (T_L) thresholds.
5. **Edge Tracking by Hysteresis** – keeps weak edges connected to strong edges; discards others.

Canny Edge Detection – Stages (Illustration)



Canny produces thin, continuous and well-localized edges with minimal noise.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 5×5 grayscale image. Apply one iteration of Canny Edge Detection (up to Non-Maximum Suppression stage) and show the results.

Given 5×5 Grayscale Image (I)

10	10	10	10	10
10	50	80	50	10
10	80	255	80	10
10	50	80	50	10
10	10	10	10	10

1. Gaussian Smoothing ($\sigma = 1.0$)

12	17	20	17	12
17	38	56	38	17
20	56	85	56	20
17	38	56	38	17
12	17	20	17	12

Noise reduced image

2. Gradient Magnitude $|G|$

0	27	25	27	0
27	65	72	65	27
25	72	0	72	25
27	65	72	65	27
0	27	25	27	0

$$|G| = \sqrt{G_x^2 + G_y^2}$$

3. Gradient Direction θ (Degrees)

–	0	0	0	–
90	45	0	135	90
90	0	–	0	90
90	135	0	45	90
–	0	0	0	–

(– : Not defined)

4. Non-Maximum Suppression

0	0	0	0	0
0	65	72	65	0
0	72	0	72	0
0	65	72	65	0
0	0	0	0	0

Thinned edges (1-pixel wide)

What does the result mean?

- Edges lie along the boundary of the bright center region.
- Non-maximum suppression keeps only local maxima in the gradient direction.
- Output is a thin edge map.

Final Answer (Up to NMS Stage): The thinned edge map after Non-Maximum Suppression is shown above.



(c) GOOGLE COLAB PYTHON PROGRAM

```
# Canny Edge Detection Example
import cv2
import numpy as np

# Given 5x5 grayscale image
I = np.array([[10, 10, 10, 10, 10],
              [10, 50, 80, 50, 10],
              [10, 80, 255, 80, 10],
              [10, 50, 80, 50, 10],
              [10, 10, 10, 10, 10]], dtype=np.uint8)

# 1. Gaussian Smoothing
blur = cv2.GaussianBlur(I, (3,3), 1.0)

# 2. Canny Edge Detection (full pipeline)
edges = cv2.Canny(I, threshold1=30, threshold2=100,
                  apertureSize=3, L2gradient=True)

# Display results
print("Original Image:\n", I)
print("\nGaussian Smoothed Image:\n", blur)
print("\nCanny Edge Output:\n", edges)
```

Output (Sample)

Original Image (5x5)

10	10	10	10	10
10	50	80	50	10
10	80	255	80	10
10	50	80	50	10
10	10	10	10	10

Gaussian Smoothed Image

12	17	20	17	12
17	38	56	38	17
20	56	85	56	20
17	38	56	38	17
12	17	20	17	12

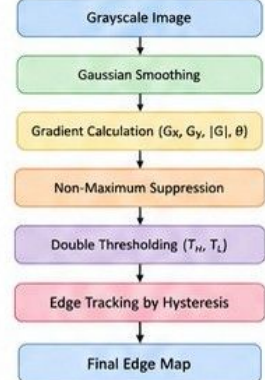
Canny Edge Output

0	0	0	0	0
0	1	1	1	0
0	1	0	1	0
0	1	1	1	0
0	0	0	0	0

(Note: Thresholds used – $T_L = 30$, $T_H = 100$)

Edges are detected around the high intensity central region.

Canny Pipeline Flow



Observation: Canny Edge Detection uses multiple stages to produce thin, well-localized and noise-free edges.

It first smooths the image, computes gradients, thins edges, applies thresholds and connects edges using hysteresis.



15. IMAGE HISTOGRAM AND HISTOGRAM EQUALIZATION



Computer Vision – Image Enhancement – Integrazation



(a) CONCEPT EXPLANATION

Q. (a) Explain image histogram and histogram equalization.

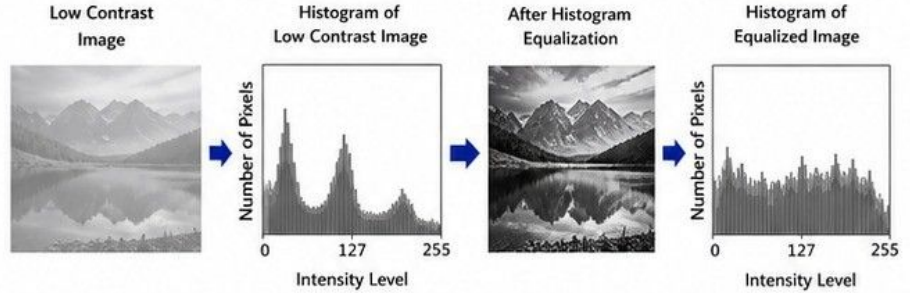
Image Histogram: A histogram shows the distribution of pixel intensities in an image. For a grayscale image with L intensity levels (0 to $L-1$), the histogram counts the number of pixels at each intensity level.

Histogram Equalization: It is a technique that improves the contrast of an image by spreading the most frequent intensity values over a wider range.

Steps:

1. Compute histogram of the image.
2. Compute probability density function (PDF).
3. Compute cumulative distribution function (CDF).
4. Compute transformation function.
5. Map original intensities using the transformation function to get equalized image.

Histogram Equalization – Process Overview



Histogram equalization redistributes intensities and improves the overall contrast.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the 4×4 grayscale image shown below. Compute its histogram, PDF, CDF, transformation function, and the histogram equalized image (use $L = 8$ intensity levels: 0 to 7).

Given 4×4 Image (I)

1	1	2	2
2	3	3	3
4	4	5	6
6	6	7	7

1. Histogram (n_k)

Intensity (k)	0	1	2	3	4	5	6	7
Count (n_k)	0	2	3	3	2	1	3	2

Total pixels, $N = 16$

2. PDF: $p_r(k) = n_k / N$

k	0	1	2	3	5	6	7
$p_r(k)$	0	0.125	0.1875	0.1875	0.0625	0.1875	0.125

3. CDF: $c_r(k) = \sum_{j=0}^k p_r(j)$

k	0	1	2	3	5	6	7
$c_r(k)$	0	0.125	0.3125	0.5000	0.6250	0.8875	1.0000

4. Transformation Function: $s_k = \text{round}[(L-1) \times c_r(k)]$ ($L = 8 \Rightarrow L-1 = 7$)

k	0	1	2	3	4	6	7
s_k	0	1	2	4	5	6	7

5. Histogram Equalized Image: replace each pixel value k by s_k

1	1	2	2
2	4	4	4
4	4	5	6
6	6	7	7

Mapping ($k \rightarrow s_k$)

0→0, 1→1, 2→2, 3→4,
4→4, 5→5, 6→6, 7→7

What does the result mean?

- Frequent intensities are spread out over a wider range.
- Contrast of the image is improved.
- Details in dark and bright regions become more visible.



(c) GOOGLE COLAB PYTHON PROGRAM

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Given 4x4 image
I = np.array([[1, 2, 2],
              [2, 3, 3],
              [4, 4, 6],
              [6, 6, 7]], dtype=np.uint8)

L = 8 # intensity levels (0 to 7)

# 1. Histogram
hist, bins = np.histogram(I, bins=np.arange(L-1), range=(0, L))

# 2. PDF
pdf = hist / L

# 3. CDF
cdf = np.cumsum(pdf)

# 4. Transformation function
T = np.round((L-1) * cdf).astype(np.uint8)

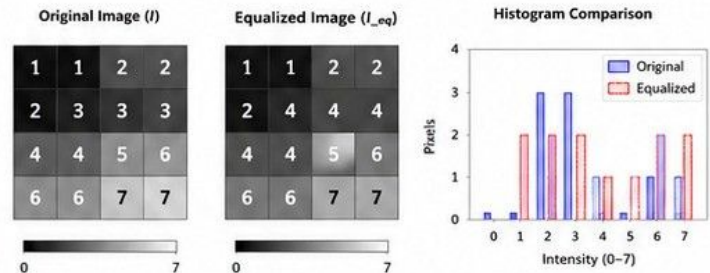
# 5. Equalized image
I_eq = T[I]

print("Histogram:", hist)
print("PDF:", pdf)
print("CDF:", np.round(cdf, 4))
print("Transformation (T):", T)
print("Equalized Image:\n", I_eq)
```

Output (Sample)

Histogram: [0 2 3 3 2 1 3 2]
PDF: [0. 0.125 0.1875 0.1875 0.125 0.0625 0.1875 0.125]
CDF: [0. 0.125 0.3125 0.5 0.625 0.6875 0.875 1.]
Transformation (T): [0 1 2 4 4 5 6 7]
Equalized Image:
[1 1 2 2]
[2 4 4 4]
[4 4 5 6]
[6 6 7 7]

Visual Illustration



Observation:

Histogram equalization redistributes the intensity values and improves contrast of the image, making it more suitable for human viewing and further processing.



16. IMAGE PYRAMIDS (GAUSSIAN PYRAMID)



Computer Vision – Image Enhancement & Multi-Scale Representation



(a) CONCEPT EXPLANATION

Q. (a) Explain the concept of image pyramids (Gaussian pyramid).

An image pyramid is a collection of images of the same scene at different resolutions arranged in a **pyramid structure**. The Gaussian pyramid is created by repeatedly applying Gaussian smoothing and down-sampling.

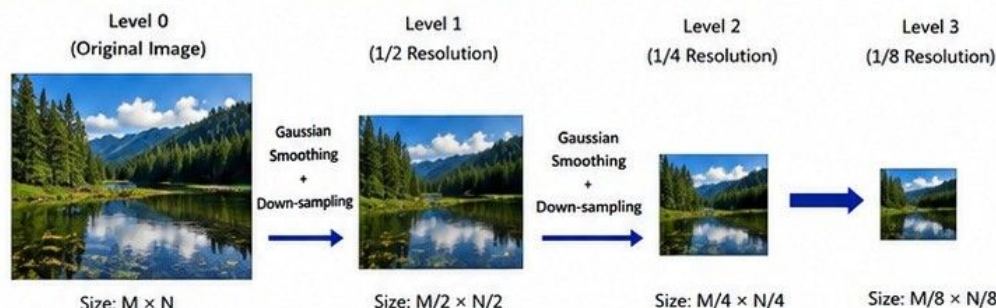
Each level is a low-pass filtered, reduced version of the previous level.

It is widely used in **image compression, object detection, optical flow, multi-scale analysis, and texture analysis**.

Steps to build Gaussian Pyramid:

1. Start with the original image as Level 0 (highest resolution).
2. Apply Gaussian filter to reduce high frequency (noise).
3. Down-sample the smoothed image by a factor of 2 (take every 2nd pixel in row and column).
4. Repeat steps 2 and 3 to get next levels.

Gaussian Pyramid – Illustration



Each level contains less detail and lower resolution.
The pyramid captures the image at multiple scales.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 4×4 grayscale image. Construct Gaussian pyramid up to Level 2 using 3×3 Gaussian kernel. Show all intermediate results.

Given 4×4 Image (Level 0)

10	20	30	40
50	60	70	80
90	100	110	120
130	140	150	160

3×3 Gaussian Kernel
(1/16)

1	2	1
2	4	2
1	2	1

Level 0 → Level 1
(1) Gaussian Smoothing (Convolution)

33.75	43.75
93.75	103.75

(2) Down-sampling by 2
(Take every 2nd pixel)

34	44
94	104

Level 1 Image
(2×2)

34	44
94	104

Calculation Notes:

- Border handling: valid region only.
- After smoothing, image size becomes 2×2.
- After down-sampling by 2, we get next level.

Level 1 → Level 2

(1) Gaussian Smoothing (Convolution)

68.75

(2) Down-sampling by 2
(Take every 2nd pixel)

69

Level 2 Image
(1×1)

69

Final Gaussian Pyramid: Level 0 (4×4) → Level 1 (2×2) → Level 2 (1×1)

Level 0 = given image, Level 1 = $\begin{bmatrix} 34 & 44 \\ 94 & 104 \end{bmatrix}$, Level 2 = [69]

What does the result mean?

- Level 0 is the original high resolution image.
- Level 1 has half the resolution (2×2).
- Level 2 has quarter the resolution (1×1).
- As level increases, image size decreases and details are reduced.



(c) GOOGLE COLAB PYTHON PROGRAM

Python Code (OpenCV)

```
import cv2
import numpy as np

# Given 4x4 grayscale image
img = np.array([[10, 20, 30, 40],
                [50, 60, 70, 80],
                [90, 100, 110, 120],
                [130, 140, 150, 160]], dtype=np.float32)

# Build Gaussian Pyramid up to Level 2
G0 = img.copy() # Level 0
G1 = cv2.pyrDown(G0) # Level 1
G2 = cv2.pyrDown(G1) # Level 2

print("Level 0 (4x4):\n", G0)
print("\nLevel 1 (2x2):\n", G1)
print("\nLevel 2 (1x1):\n", G2)
```

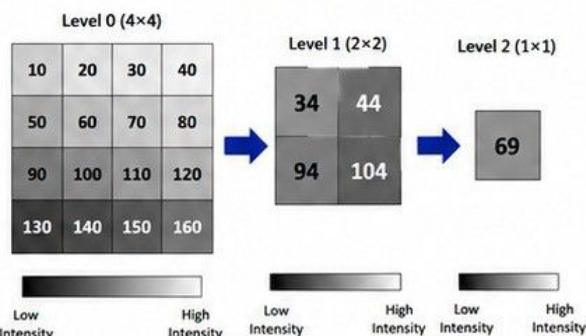
Output (Sample)

```
Level 0 (4x4):
[[ 10.  20.  30.  40.]
 [ 50.  60.  70.  80.]
 [ 90. 100. 110. 120.]
 [130. 140. 150. 160.]]

Level 1 (2x2):
[[ 34.  44.]
 [ 94. 104.]]

Level 2 (1x1):
[[ 69.]]
```

Visual Illustration



Gaussian Pyramid Flow



Observation:

Gaussian pyramid represents the image in multiple scales. It helps in analyzing images at different resolutions, reduces computation, and is the basis for many computer vision algorithms.



17. HOUGH TRANSFORM (LINE DETECTION)



Computer Vision – Feature Extraction – Shape Representation

(a) CONCEPT EXPLANATION

Q. (a) Explain the Hough Transform for line detection.

The Hough Transform is a feature extraction technique used to detect lines (or other shapes) in an image.

It works in a parameter space (ρ - θ space) instead of the image space. Each edge point (x, y) votes for all possible lines passing through it.

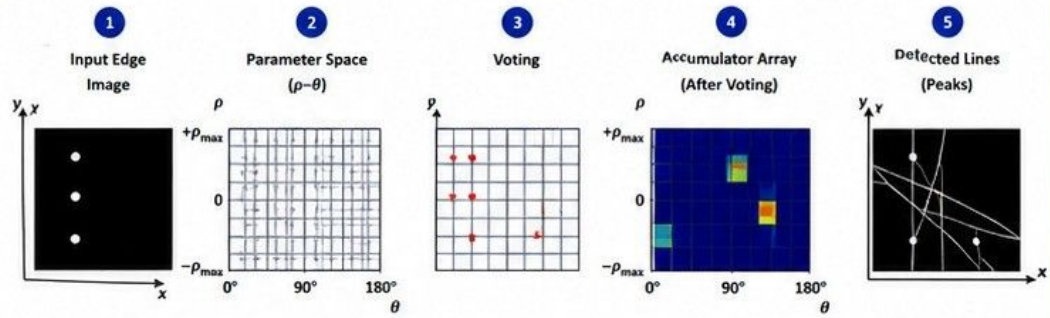
Line equation in polar form:

$$\rho = x \cos \theta + y \sin \theta$$

Where,

- ρ = perpendicular distance from origin to the line
- θ = angle between x -axis and the perpendicular
- (x, y) = edge point in image space

How it Works



Peaks in the accumulator array correspond to lines in the image.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given edge points: $(2,1)$, $(2,3)$, $(2,5)$ in an image. Use Hough Transform to detect a line.
Let $\theta = \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}$ and ρ ranges from -5 to 5 with step 1.

1. Edge Points

2. Compute $\rho = x \cos \theta + y \sin \theta$

3. Accumulator Array (ρ vs θ)

Point	x	y
P_1	2	1
P_2	2	3
P_3	2	5

θ	$\cos \theta$	$\sin \theta$	ρ values for each point		
			$P_1 (2,1)$	$P_2 (2,3)$	$P_3 (2,5)$
0°	1	0	2	2	2
45°	0.707	0.707	2.12	3.54	4.95
90°	0	1	1	3	5
135°	-0.707	0.707	-0.71	0.71	2.12

ρ values:	-5	-4	-3	-2	-1	0	+2	+3	+4	+5
θ	0	0	0	0	0	2	0	0	0	0
45°	0	0	0	0	1	1	1	0	1	0
90°	0	0	0	0	1	0	1	0	1	0
135°	0	0	0	0	1	1	0	1	0	0

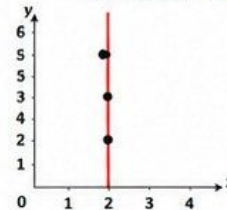
4. Interpretation

- Peak value 2 at $(\theta = 0^\circ, \rho = 2) \rightarrow$ Strong line detected.
- This corresponds to vertical line: $\rho = 2$.
- All given points lie on line $x = 2$.

5. Detected Line

Maximum peak at $(\theta = 0^\circ, \rho = 2)$
Line equation: $\rho = 2$
Since $\theta = 0^\circ \Rightarrow \rho = x$
Hence, $x = 2$ (vertical line)

Detected Line on Image



What does the result mean?

- The accumulator peak represents the most likely line passing through the edge points.
- Hough Transform is robust to noise and works even if points on the line are not continuous.
- It is widely used for lane detection, road markings, building edges, etc.

Final Answer: Detected line is $x = 2$ ($\rho = 2, \theta = 0^\circ$)



(c) GOOGLE COLAB PYTHON PROGRAM

Python Code (OpenCV)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create blank image and draw sample edge points
img = np.zeros((100,100), dtype=np.uint8)
points = [(2,1), (2,3), (2,5)]
for (x,y) in points:
    cv2.circle(img, (int(x*10), int(y*10)), 2, 255, -1)

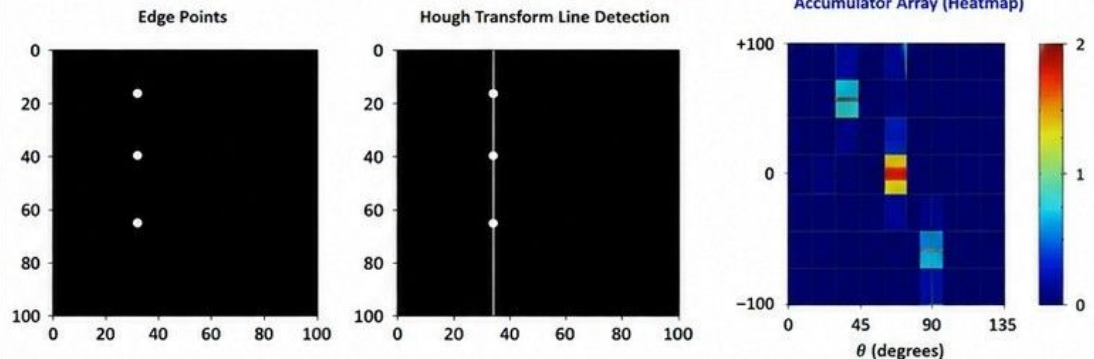
# Hough Transform Parameters
theta = np.deg2rad(np.arange(0,180,45))
rho_max = int(np.hypot(100,100))
rho = np.arange(-rho_max, rho_max)

# Hough Line Transform
lines = cv2.HoughLines(img, 1, np.pi/180, threshold=2)

# Draw detected lines
if lines is not None:
    for line in lines:
        rho_v, theta_v = line[0]
        a = np.cos(theta_v); b = np.sin(theta_v)
        x0 = a*rho_v; y0 = b*rho_v
        pt1 = (int(x0 + 1000*(a-b)), int(y0 + 1000*(a)))
        pt2 = (int(x0 - 1000*(a-b)), int(y0 - 1000*(a)))
        cv2.line(img, pt1, pt2, 255, 1)

# Display result
plt.imshow(img, cmap='gray')
plt.title('Hough Transform Line Detection')
plt.axis('on')
plt.show()
```

Output (Sample)



Observation: Hough Transform maps edge points to a parameter space and finds peaks (ρ, θ) that correspond to lines. It is effective even with noisy or incomplete data.

18. IMAGE SEGMENTATION USING THRESHOLDING

Computer Vision – Image Segmentation – Region Separation



(a) CONCEPT EXPLANATION

Q. (a) Explain image segmentation using thresholding.

Image thresholding is a simple and effective technique used to segment an image into foreground (object) and background based on intensity.

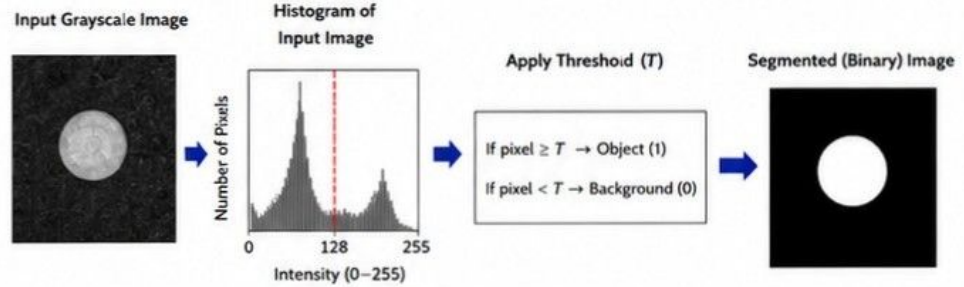
A threshold value T is chosen. Pixels with intensity greater than or equal to T are assigned to the object (1) and others to the background (0).

Mathematical Expression

$$g(x, y) = \begin{cases} 1, & \text{if } f(x, y) \geq T \\ 0, & \text{if } f(x, y) < T \end{cases}$$

Where, $f(x, y)$ = input image, $g(x, y)$ = segmented image

How Thresholding Works



- Thresholding is easy to implement and computationally efficient.
- Best suited for images with good contrast between object and background.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 5x5 grayscale image and threshold value $T = 128$, perform binary thresholding.

1. Input Grayscale Image ($f(x, y)$)

52	80	120	130	200
30	90	110	160	210
15	60	100	150	220
40	70	125	170	240
20	55	115	180	250

2. Apply Thresholding Rule
($T = 128$)

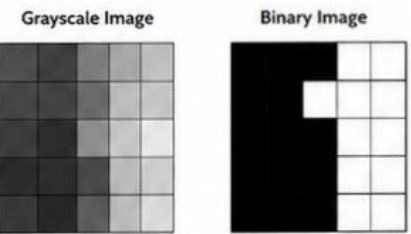
If pixel $\geq 128 \rightarrow 1$
If pixel $< 128 \rightarrow 0$

3. Binary Segmented Image ($g(x, y)$)

0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1

4. Interpretation

- Pixels with intensity ≥ 128 are set to 1 (white).
- Pixels with intensity < 128 are set to 0 (black).
- The object (bright region) on the right side is successfully separated from the background.



Final Answer: The binary segmented image after thresholding with $T = 128$ is shown above.



(c) GOOGLE COLAB PYTHON PROGRAM

Python Code (OpenCV)

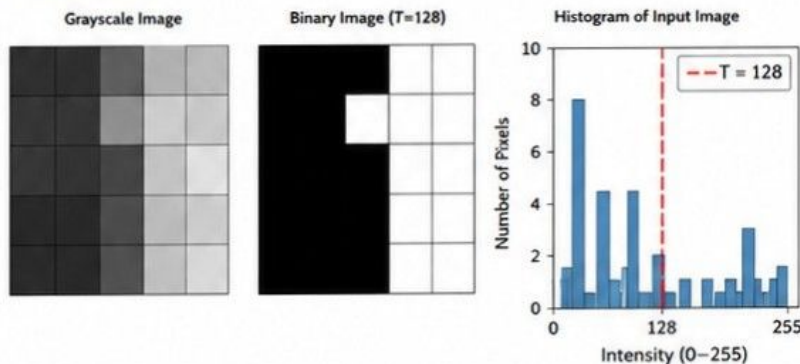
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Given 5x5 grayscale image
img = np.array([
    [52, 80, 120, 130, 200],
    [30, 90, 110, 160, 210],
    [15, 60, 100, 150, 220],
    [40, 70, 125, 170, 240],
    [20, 55, 115, 180, 250]
], dtype=np.uint8)

T = 128 # Threshold value
# Apply binary thresholding
_, binary = cv2.threshold(img, T, 255, cv2.THRESH_BINARY)

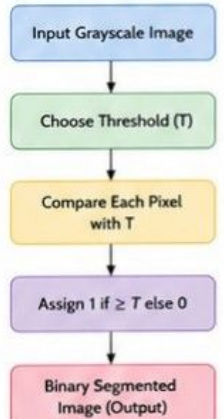
# Display results
plt.figure(figsize=(12,4))
plt.subplot(1,3,1), plt.title('Grayscale Image'), plt.imshow(img, cmap='gray'),
plt.subplot(1,3,2), plt.title('Binary Image'), plt.imshow(binary, cmap='gray'),
plt.subplot(1,3,3), plt.title('Histogram of Input Image'), plt.hist(img.ravel(), bins=256, range=[0,255], color='steelblue')
plt.xlabel('Intensity (0-255)'), plt.ylabel('Number of Pixels')
plt.legend(), plt.tight_layout()
plt.show()
```

Output (Sample)



In the histogram, the threshold $T = 128$ separates the lower intensity (background) from higher intensity (object) pixels.

Thresholding Pipeline



Observation: Thresholding is a fundamental segmentation technique that converts a grayscale image into a binary image by separating object and background using a threshold value.



19. MORPHOLOGICAL OPERATIONS (DILATION & EROSION)



Computer Vision – Image Processing – Shape Analysis



(a) CONCEPT EXPLANATION

Q. (a) Explain morphological operations: Dilation and Erosion.

Morphological operations process images based on shapes. They use a small matrix called Structuring Element (SE) to probe and modify the shape of objects.

1. Dilation ($\oplus$):

Adds pixels to the boundaries of objects. It makes objects thicker and fills small holes.

2. Erosion ($\ominus$):

Removes pixels from the boundaries of objects. It makes objects thinner and removes small noises.

Let A be input binary image and B be structuring element.

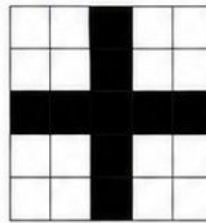
• **Dilation:** $A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\}$

• **Erosion:** $A \ominus B = \{z \mid B_z \subseteq A\}$

Where $\hat{B}$ is reflection of B about its origin.

How Dilation and Erosion Work (Example)

Input Binary Image (A)

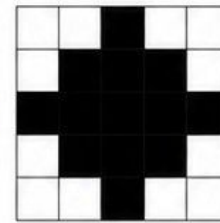


Structuring Element (B)

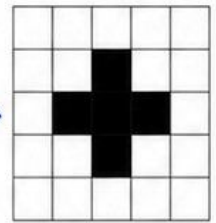


(3x3 Square SE)

Dilation ($A \oplus B$)



Erosion ($A \ominus B$)



• Dilation expands object regions.

• Erosion shrinks object regions.

• Dilation is useful for filling holes and connecting objects.

• Erosion is useful for removing noise and separating objects.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 5x5 binary image A and 3x3 structuring element B (all ones).

Perform (i) Dilation ($A \oplus B$) and (ii) Erosion ($A \ominus B$).

1. Input Binary Image A

0	0	1	0	0
0	1	1	1	0
0	1	1	1	0
0	0	1	0	0
0	0	0	0	0

2. Structuring Element B (3x3)

1	1	1
1	1	1
1	1	1

(Origin at center)

3. Dilation ($A \oplus B$)

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
0	1	1	1	0

4. Erosion ($A \ominus B$)

0	0	0	0	0
0	0	1	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

Final Answer: Dilation ($A \oplus B$) and Erosion ($A \ominus B$) results are shown above.

5. Interpretation

- Dilation adds pixels around the object boundary.
- Erosion removes boundary pixels and shrinks the object.
- Dilation filled the gaps and connected the object.
- Erosion removed outer layers and kept only core.



(c) GOOGLE COLAB PYTHON PROGRAM

Python Code (OpenCV)

```
import cv2
import numpy as np

# 1. Input binary image A (5x5)
A = np.array([[0,0,1,0,0],
              [0,1,1,1,0],
              [0,1,1,1,0],
              [0,0,1,0,0],
              [0,0,0,0,0]], dtype=np.uint8)

# 2. Structuring Element B (3x3 ones)
B = np.ones((3,3), dtype=np.uint8)

# 3. Dilation and Erosion
A_dilate = cv2.dilate(A, B, iterations=1)
A_erode = cv2.erode(A, B, iterations=1)

# 4. Display results
print("Input Image A:\n", A)
print("Dilation A ⊕ B:\n", A_dilate)
print("Erosion A ⊖ B:\n", A_erode)
```

Output (Sample)

Input Image A

0	0	1	0	0
0	1	1	1	0
0	1	1	1	0
0	0	1	0	0
0	0	0	0	0

Dilation $A \oplus B$

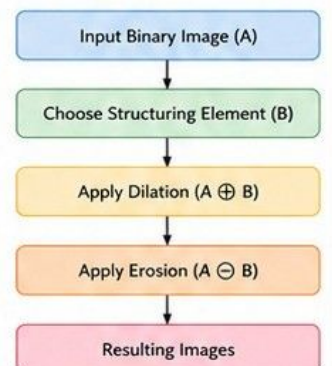
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
0	1	1	1	0

Erosion $A \ominus B$

0	0	0	0	0
0	0	1	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

Note: For border pixels, zero-padding is used by default in OpenCV.

Morphological Operations Pipeline



Observation: Morphological operations are powerful tools in shape analysis, noise removal, object detection and preprocessing in computer vision.

Applications:

- Removing noise
- Connecting components
- Filling holes
- Boundary extraction

20. IMAGE SMOOTHING USING AVERAGING FILTER

Computer Vision – Image Enhancement – Noise Reduction

(a) CONCEPT EXPLANATION

Q. (a) Explain image smoothing using averaging filter.

Image smoothing (or blurring) is used to reduce noise and small intensity variations in an image. Averaging filter (Mean filter) replaces each pixel value by the average of its neighboring pixels.

Mathematical Expression

$$g(x, y) = \frac{1}{mn} \sum_{i=-k}^k \sum_{j=-l}^l f(x+i, y+j)$$

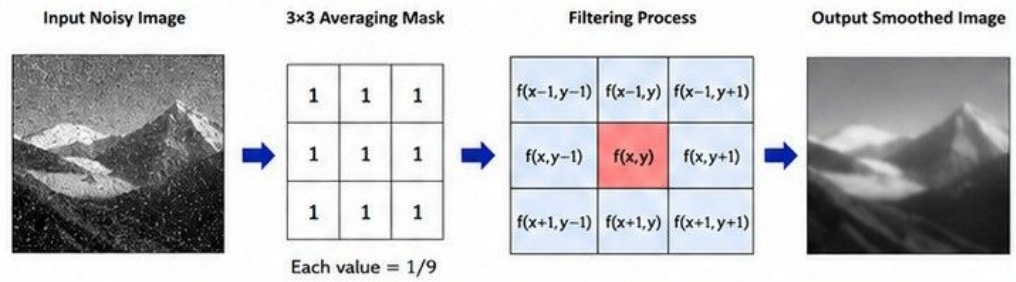
Where,

$f(x, y)$ = input image

$g(x, y)$ = filtered (smoothed) image

$m \times n$ = size of the filter mask ($m = 2k + 1$, $n = 2l + 1$)

How Averaging Filter Works (Example with 3×3 Mask)



Key Points

- Reduces noise and minor variations.
- Simple and effective.
- Preserves overall brightness.
- Larger mask size gives more smoothing.

(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 5×5 grayscale image. Apply 3×3 averaging filter to compute the smoothed image.

1. Input Image (f)

10	20	30	40	50
15	25	35	45	55
20	30	40	50	60
25	35	45	55	65
30	40	50	60	70

2. 3×3 Averaging Mask (Mean Filter)

1	1	1
1	1	1
1	1	1

Each value = 1/9

3. Calculation (Example for center pixel position (3,3))

Neighborhood around (3,3):

25	35	45
30	40	50
35	45	55

Average = sum / 9

$$= (25+35+45+30+40+50+35+45+55) / 9$$

$$= 360 / 9 = 40$$

So, output pixel $g(3,3) = 40$

4. Smoothed Image (g) (after applying 3×3 averaging filter)

20	30	40
30	40	50
40	50	60

Note: Since a 3×3 mask cannot be centered on border pixels, the output size becomes $(5-3+1) \times (5-3+1) = 3 \times 3$.

Final Answer: The smoothed image using 3×3 averaging filter is shown above.

Interpretation

- The averaging filter smooths the image.
- Sharp changes are reduced.
- Noise and small details are removed.

(c) GOOGLE COLAB PYTHON PROGRAM

Python Code (OpenCV + NumPy)

```
import cv2
import numpy as np

# Given 5x5 grayscale image
img = np.array([[10, 20, 30, 40, 50],
                [15, 25, 35, 45, 55],
                [20, 30, 40, 50, 60],
                [25, 35, 45, 55, 65],
                [30, 40, 50, 60, 70]], dtype=np.float32)

# 3x3 Averaging (Mean) filter
kernel = np.ones((3,3), np.float32) / 9.0

# Apply filter
smoothed = cv2.filter2D(img, -1, kernel, borderType=cv2.BORDER_CONSTANT)

# Show results
print("Input Image:\n", img.astype(np.uint8))
print("\nSmoothed Image:\n", smoothed.astype(np.uint8))
```

Output (Sample)

Input Image (5×5)

10	20	30	40	50
15	25	35	45	55
20	30	40	50	60
25	35	45	55	65
30	40	50	60	70

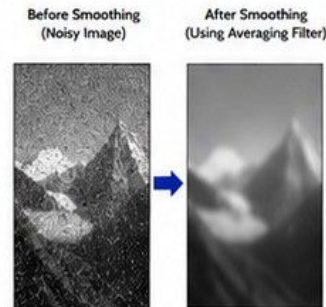
Smoothed Image (3×3)

20	30	40
30	40	50
40	50	60

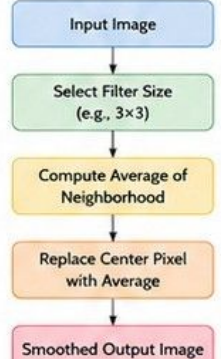
Note:

- Border pixels are not computed with a 3×3 mask (valid convolution).
- Use larger masks (e.g., 5×5, 7×7) for stronger smoothing.

Visual Illustration



Averaging Filter Pipeline



Observation: Averaging filter is an effective low-pass filter.

It reduces noise but may blur edges if the mask size is large.

Applications:

- Noise reduction
- Preprocessing
- Smoothing textures
- Improves image quality